

MCP Security Methodology

A framework for static security analysis of Model Context Protocol servers.

Compuute AB — v1.0 — 2026-05-23

Abstract

The Model Context Protocol (MCP) reached production adoption at 78% of enterprise AI teams by Q1 2026 (DigitalApplied 2026). With 17 468 indexed MCP servers and the protocol now declared an industry standard by Anthropic, OpenAI, Google, and Microsoft, the attack surface has multiplied without a corresponding maturity in MCP-specific security tooling. Generic SAST tools (Snyk, Aikido, Semgrep) miss the MCP-specific threat patterns that arise from the agent-to-tool interaction model.

This paper describes the methodology behind `compuute-scan` (open-source, MIT) and its hosted `compuute-scan-api` service. We document the threat model we test against, the rule taxonomy, our false-positive transparency, and the manual triage process layered on top. Our goal is not to claim precision we do not have, but to publish what static analysis can and cannot prove — so consumers (human or autonomous agent) can decide what triage capacity they need.

1. Threat model

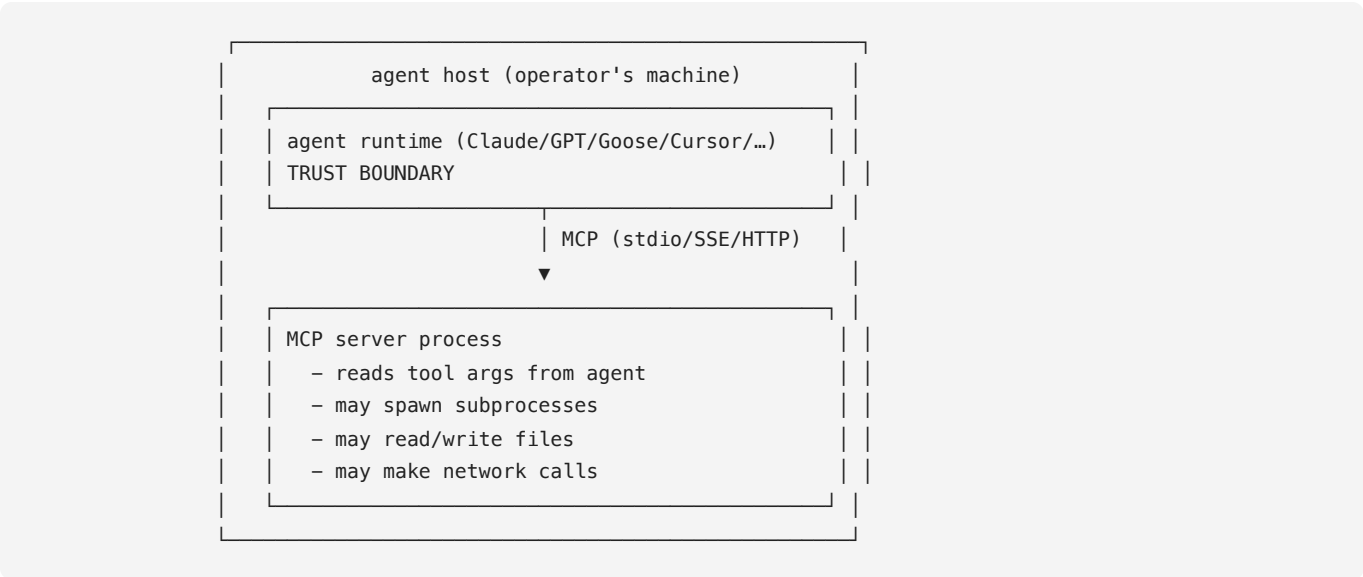
We use STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) adapted for the MCP-server attack surface.

1.1 Adversaries

Adversary	Capability
A1 — Tool-description attacker	Publishes an MCP server whose tool descriptions contain prompt-injection payloads aimed at the calling agent
A2 — Argument-injection attacker	Submits crafted MCP tool arguments designed to be interpreted as flags by a sub-process the server spawns

Adversary	Capability
A3 — Supply-chain attacker	Compromises a dependency that an MCP server installs at startup or runtime
A4 — Network attacker	Intercepts MCP traffic at the transport layer (mitigated by TLS but worth modeling)
A5 — Resource-exhaustion attacker	Sends inputs designed to consume disproportionate CPU, memory, or disk on the MCP server host

1.2 Trust boundaries



Compute-scan analyzes the **MCP server process** code statically, before deployment.

1.3 Threat-to-rule mapping

Adversary	Primary patterns we detect
A1 (prompt injection)	Out of scope for static analysis — covered by separate agent-side defenses
A2 (argument injection)	L1-002 (shell exec), L1-038 (runner-binary args), L1-018/024/033 (Go/Rust/Java equivalents)
A3 (supply-chain)	L0 dependency pinning, L0-CVE offline DB, license compliance
A4 (network)	L1-007 SSL/TLS bypass, L1-022 CORS wildcard
A5 (resource exhaustion)	L1-029 ReDoS, L1-013 unbounded file read

2. Rule taxonomy

`compute-scan` organizes detection rules into layers borrowed from defense-in-depth modeling: L0 (discovery), L1 (sandboxing), L2 (authorization), L3 (tool integrity), L4 (monitoring).

The open-source CLI ships **L0 + L1** (37 active rules + 2 negative checks + 3 dependency checks). L2–L4 are reserved for the manual audit tier at compute.se/audit.

2.1 Layer summary

Layer	Focus	Examples
L0	Discovery — what does the server expose, what does it depend on?	Transport detection, tool inventory, dependency pinning, license compliance, offline CVE matching (40+ top packages)
L1	Sandboxing — does the code respect process isolation?	<code>eval</code> , <code>exec.Command</code> , <code>pickle.loads</code> , runner-binary arg injection (L1-038), CORS wildcards, SSL bypass
L2	Authorization (paid)	RBAC implementation, secret handling, JWT validation, PII/GDPR handling
L3	Tool integrity (paid)	SSRF, SQL injection in tool implementations, prompt-injection-resistance review
L4	Monitoring (paid)	Audit logging completeness, rate-limit implementation, error-leakage review

2.2 Why broad detection at L0 + L1

L0 and L1 are designed to **flag patterns**, not to **prove exploits**. A vulnerable pattern is necessary for an exploit but not sufficient — the pattern is reachable from attacker-controlled input only some of the time, and the runtime may have compensating controls (sandboxes, capability restrictions, sanitizers) that static analysis cannot see.

We chose broad detection because:

1. **Missing a critical is worse than flagging an FP.** An MCP server that ships with `eval(toolArgs.code)` and gets exploited is worse than one that triggers a triage queue.
2. **Triage capacity is fungible.** A buyer who needs precision can apply manual review; a scanner that hides FPs cannot retrofit recall.
3. **Disclosure shifts the contract.** Every response carries a `_disclaimer` field stating that findings are pattern matches. A consumer who displays the disclaimer correctly never presents findings as confirmed vulnerabilities.

2.3 Language coverage

Language	MCP SDK source	Status
TypeScript	Anthropic (official)	Full coverage
JavaScript	Anthropic	Full coverage
Python	Anthropic (official)	Full coverage
Go	Anthropic (official)	Full coverage
Rust	Anthropic (official)	Full coverage
C# / .NET	Microsoft (official, March 2026)	Full coverage
Java / Kotlin	Spring (official, March 2026)	Full coverage

`compute-scan` v0.6.2 was the first MCP-specific scanner to ship .NET + Java/Kotlin coverage, aligned with the official Microsoft and Java/Spring SDK releases.

3. Detection methodology

Each rule is a small Python object: `id`, `title`, `severity`, `owasp`, `cwe`, `capec`, `nis2`, `description`, `recommendation`, `test(line)` predicate, and a list of `guards` (regex patterns that, when present near a hit, suppress the finding).

3.1 Predicate-and-guard model

```
{
  id: 'L1-038',
  title: 'MCP runner-binary argument injection (npv/uvx/pipx/pnpv)',
  severity: 'high',
  cwe: 'CWE-88',
  owasp: 'A03:2021 Injection',
  test: (line) => {
    // skip comments
    if (/^\s*\/\//.test(line) || /^\/\s*\/.test(line)) return false;
    // match spawn-family call with a runner binary literal as first arg
    const spawnRunner = /\b(?:spawn|spawnSync|execFile)\s*(\s*['"](?:npv|uvx|pipx|pnpv)['"])/;
    if (!spawnRunner.test(line)) return false;
    // flag if args are variable (not literal) or template literal
    return hasVariableArg || hasTemplateInLine || !hasArrayLiteral;
  },
  guards: [
    /--package=['"][\w@\/\.-]+@[\/d.]+/, // pinned --package=name@version
    /\s\.startsWith\s*(\s*['"]-['"]\s*)/, // arg.startsWith('-') flag-aware guard
    /shell\s*:\s*false/, // explicit shell:false
  ]
}
```

```
1,  
},
```

A hit is reported when the predicate matches AND no guard pattern is present within the configured `GUARD_WINDOW` (default ± 15 lines).

3.2 Guard windows trade precision for recall

Guards run within a 15-line window above and below the hit. This window is small enough to keep performance fast (one regex per line in a tight loop) and large enough to catch most local sanitization patterns. The trade-off is that **defense-in-depth one module away — e.g., a sanitizer called in the parent function** — is invisible to the scanner. This is the source of most false positives in our historical FP data (see Section 5).

3.3 Negative rules

In addition to per-line rules, `compute-scan` runs whole-codebase negative checks:

- L1-011 — "no security-headers middleware detected anywhere" (medium)
- L1-027 — "no rate limiting detected" (medium)

These run after the per-line pass and produce a single finding scoped to "(entire codebase)" when triggered.

3.4 Self-test discipline

Every rule has a positive test fixture (a small file that should fire) and a negative test fixture (a similar file that should NOT fire). The negative fixture is critical — without it, a rule could become arbitrarily broad without anyone noticing. We currently average 2 fixtures per rule; the P1 backlog target is ≥ 10 per rule.

4. False-positive transparency

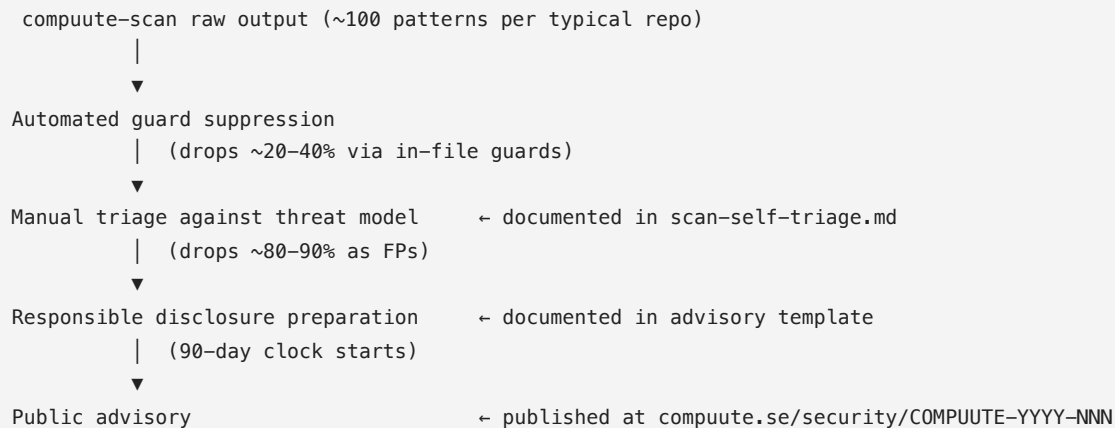
A scanner that hides its FP rate sells precision it doesn't have. Our published FP behaviour is documented in `docs/FP-RATES.md`. Summary:

- **~90% raw FP rate** on production MCP repos without triage (138 raw → 13 confirmed on modelcontextprotocol/servers, March 2026).
- **~10% confirmed-finding rate** after manual validation.
- Per-rule rates published in `docs/FP-RATES.md` with sample size.

We measure FP against the **Ox Security threat model for MCP servers**, not against "what would Snyk find?" The two scanners answer different questions.

5. Validation pipeline

Findings flow through three filters before they reach a public advisory.



5.1 Manual triage criteria

A raw finding becomes a confirmed finding only if all four are met:

1. The vulnerable pattern is present (scanner already confirmed)
2. The input that reaches the pattern is **attacker-controllable** (requires reading the code path; not derivable from scanner alone)
3. The attack chain reaches an **impact** the threat model takes seriously (RCE, secret exfiltration, privilege escalation)
4. No **compensating control** breaks the chain (sandbox, capability restriction, allow-listing, etc.)

5.2 Verification rule

We apply a "verification before acceptance" rule to ourselves (see `~/claude/CLAUDE.md`):

"A technical claim about code may not be accepted or forwarded until it has been run against the concrete target. Claim that a rule catches X → show the line it catches. Claim that a change fixes Y → run before/after against the same input, show the diff in outcome,

not in code. Claim about FP/TP → read each hit against the threat model, one row per verdict."

This applies to ourselves, to reviewers, and to other agents. Confident-but-unverified claims have been a source of credibility damage in security tooling historically (see the well-documented Snyk vs Socket vs WhiteSource positioning fights of 2022–2024).

6. Disclosure process

We follow a 90-day coordinated disclosure window, modeled on Project Zero.

Day	Phase
0	Receipt acknowledged, severity classification (CVSS v4)
1–7	Triage; reporter contacted; advisory drafted
7–60	Vendor coordination; fix implemented + validated
60–90	Public-advisory wording; CVE request via MITRE if applicable
90	Public release at <code>compute.se/security/COMPUUTE-YYYY-NNN</code> ; or earlier if fix is live and reporter agrees

Advisories are numbered `COMPUUTE-YYYY-NNN` . The first one, **COMPUUTE-2026-001** (trycua/cua npx-argument injection), entered the disclosure window on 2026-05-23.

7. Differentiation from generic SAST

We are not Snyk. We are not Aikido. We are not Socket. We do not compete on:

- Number of supported languages outside MCP-SDK scope (Snyk: 30+; us: 8 MCP-SDK languages)
- Size of CVE database (Snyk: 5M+ packages; us: 40 packages curated for high-impact)
- Mature SBOM tooling (Snyk: enterprise SBOM with attestation; us: CycloneDX 1.5 via cyclonedx-bom)
- Enterprise procurement support (Snyk: SOC 2 Type II, ISO 27001; us: working toward Type I)

We compete on **MCP-specific rule depth** and **threat-intel response cadence**. The first one was demonstrated with L1-038 (rule added within one week of the Ox Security publication). The second is the audit pipeline at compute.se/audit.

8. Open questions and roadmap

8.1 Known limitations

- L1-009 multi-line-import false-positive class (upstream regex over-match; documented in `docs/FP-RATES.md`)
- No dataflow analysis (L2-tier; manual today; opt-in mode in a future major release)
- Guard window of ± 15 lines misses cross-function defense (architectural limit)
- 90% raw FP rate is what we measure today; methodology improvements (per-rule fixture set, confidence fields) are P1+ backlog items

8.2 Roadmap

- **v0.7** — confidence fields per finding, per-rule unit-test coverage
 - **v0.8** — opt-in dataflow analysis for top-5 rules
 - **v0.9** — SARIF format output for native GitHub Code Scanning integration
 - **v1.0** — first stable release with locked rule taxonomy and SLSA L3 attestation on all release artifacts
-

9. Appendix A — references

Source	Why
Anthropic — Model Context Protocol specification	Protocol baseline
Ox Security — npx argument injection research, May 2026	Inspired L1-038
OWASP Top 10 — 2021	Severity classifications
MITRE CWE database — CWE-88, CWE-22, CWE-78, CWE-94	Standardised weakness IDs
MITRE CAPEC — CAPEC-88 et al.	Attack pattern classifications
EU NIS2 directive, Article 21(2)(e)	Compliance mapping
Sonatype 2026 State of Software Supply Chain	Industry threat baseline

Source	Why
Practical-DevSecOps SLSA Framework Guide 2026	Build-track attestation

10. Appendix B — how to read a scan response

A typical response from `POST /v1/scan` contains:

Field	Read it as
<code>score</code>	Coarse 0–100 indicator. Treat as triage prioritization, not as confidence
<code>summary.critical</code> , <code>summary.high</code> , etc.	Raw severity counts before manual triage
<code>top_findings[]</code>	Top-10 most severe findings with file+line — investigation entry points
<code>recommendation</code>	Human-readable action guidance ("AVOID", "REVIEW", "CLEAN")
<code>_disclaimer</code>	Mandatory — the pattern-match disclaimer. Display this to your users
<code>scanner.version</code>	Audit field — proves which rule set produced the findings
<code>performance.scan_seconds</code>	Should be <5s for repos <100 files; longer indicates a possible scan path bug

A consumer that surfaces `_disclaimer` to its end user and treats `top_findings` as "things to look at" rather than "things you have" will get the most out of the scanner.

End of methodology v1.

This document will be re-released as v2 when L2 dataflow analysis lands, or 6 months from today, whichever comes first. Comments and corrections: daniel@compute.se.